```
import Mathlib.Tactic

open Set Classical


-- # §1: Sets

open scoped Set
section Definitions


-- **An error**
example (S : Set) := sorry
example {α : Type} (S : Set α) : S = S := by
  rfl

-- `⌘`

-- **A tautology**

example (α : Type) (x : α) (S : Set α) : x ∈ S ↔ S x := by
  rfl


-- **The positive integers**

def PositiveIntegers : Set ℤ := by
  -- intro d
  --  use if 0 < d then True else False
  -- *or* exact (fun d ↦ 0 < d)
  -- *or* exact (0 < ·) d
  exact (0 < ·)


lemma one_posint : 1 ∈ PositiveIntegers := by
  -- unfold PositiveIntegers
  -- rw [Set.mem_def]
  -- have := Nat.one_pos
  -- exact this -- *why does this fail?*
  -- rw [← Int.ofNat_lt] at this
  -- exact this
```

```
42    -- *A better proof*
43    exact Int.zero_lt_one
44
45  def PositiveNaturals : Set ℕ := by
46    exact (0 < ·)
47
48  example : 1 ∈ PositiveNaturals := by
49    -- apply one_pos -- It *fails*!
50    -- exact @one_pos ℕ _ _ _ _ _
51    -- apply @one_pos ℕ
52    exact Nat.zero_lt_of_ne_zero Nat.one_ne_zero
53
54  -- Why does this *fail*? How to fix it?
55  -- example : (-1) ∉ PositiveNaturals := sorry
56  /- It fails because Lean cannot be convinced that `-1 : ℕ`.
    The best we can do is-/
57  example : (-1) ∉ PositiveIntegers := by
58    intro h
59    replace h := h.out --not "really" needed, but sometimes
    useful
60    -- grind -- a nice tactic that can prove these "linear
    (in)equalities"
61    exact (Int.negSucc_not_nonneg (0 + 0).succ).mp (by exact
    h) --can be found by `exact?`
62
63  -- **The even naturals**
64
65  def EvenNaturals : Set ℕ := by
66    -- intro d
67    -- exact if d % 2 = 0 then True else False
68    exact (· % 2 = 0)
69
70  example (n : ℕ) : n ∈ EvenNaturals → (n + 2) ∈ EvenNaturals
    := by
71    intro h
72    replace h := h.out
73    -- rw [Nat.add_mod_right]-- a pity it does not work...
74    -- rw [mem_def]
75    rw [← Nat.add_mod_right] at h -- try to comment the
    `replace` three lines above
76    exact h
77
```

```
78
79  -- **An abstract set**
80
81  def AbstractSet {α : Type} (P : α → Prop) : Set α := P
82  def AbstractSet' {α : Type} (P : α → Prop) : Set α := setOf
    P
83
84  /- The same, but it is a general principle that the second
    version is better because it
85  avoids abusing `defeq`. -/
86  example {α : Type} (P : α → Prop) : AbstractSet P =
    AbstractSet' P := by
87    rfl
88
89
90  -- `⌘`
91
92
93  -- **Subsets as implication**
94  example {α : Type} (S T : Set α) (s : α) (hST : S ⊆ T) (hs :
    s ∈ S) : s ∈ T := by
95    apply hST
96    exact hs
97
98  -- **A double inclusion**
99
100 example (α : Type) (S T W : Set α) (hST : S ⊆ T) (hTW : T ⊆
    W) : S ⊆ W := by
101   intro s hs
102   apply hTW
103   apply hST
104   exact hs
105   -- *An alternative proof*
106   -- intro s hs
107   -- exact hTW <| hST hs
108   -- *Another one*
109   -- exact hTW (hST hs)
110
111 -- **Another take on subsets and sets as types**
112
113 def subsub {α : Type} {S : Set α} (P : S → Prop) : Set (S :
    Type) := P
```

```
114
115 def subsub' {α : Type} {S : Set α} (P : α → Prop) : Set (S :
    Type) := by
116   intro a
117   exact P a
118
119 -- Are they *equal*? This is an exercise below.
120
121 -- Why does this *fail*? How to fix it?
122 -- example (α : Type) (S : Set α) (P : S → Prop) (x : ↑S)
    (hx : x ∈ subsub P) : x ∈ S := sorry
123 /- *Sol.:* This fails because `x` is of type `↑S`, but `S`
    is in `Set α`, so only terms of type `α`
124 can be tested for membership in `S`. It can be made to work
    as follows:-/
125 example (α : Type) (S : Set α) (P : S → Prop) (x : ↑S) (hx :
    x ∈ subsub P) : {s : S // P s} := by
126   use x
127   exact hx
128
129 -- **More about this "injection"  `Set α ↪ Type`?**
130
131 -- Why does this *fail*? How to fix it?
132 -- example : ∀ n : PositiveIntegers, 0 ≤ n := sorry
133 /- *Sol.:*  This fails because `0` is a term of `ℕ`, whereas
    `n` is a term of `PositiveIntegers`.
134 They cannot be compared directly, because `n` is actually a
    *pair* of a natural number and a proof
135 of its positivity. It can be made to work as follows-/
136 example : ∀ n : PositiveIntegers, 0 < n.1 := by
137   rintro ⟨-, hn⟩ --use first `rintro ⟨n, hn⟩` and then
    `rintro ⟨_, hn⟩`
138   exact hn
139
140
141 -- `⌘`
142
143 end Definitions
144
145 section Operations
146
147 -- ## Operations on sets
```

```
148
149 -- **Self-intersection is the identity, proven with
    extensionality**
150
151 example (α : Type) (S : Set α) : S ∩ S = S := by
152   -- ext
153   -- constructor
154   -- · intro h --rintro ⟨h, h⟩
155   --   exact h.1 -- exact h
156   -- · intro h
157   --   exact ⟨h, h⟩
158 -- *Alternative proof*
159   ext
160   rw [← eq_iff_iff]
161   exact and_self _
162
163 -- **The union**
164
165 example (α : Type) (S T : Set α) (H : S ⊆ T) : S ∪ T = T :=
    by
166   ext x
167   rw [Set.subset_def] at H
168   exact or_iff_right_of_imp (H x)
169
170
171 -- **An _unfixable_ problem**
172
173 -- example (α β : Type) (S : Set α) (T : Set β) : S ⊆ S ∪ T
    := sorry
174 /- *Sol.:*  Well, it was unfixable, so there is no
    solution...-/
175
176
177 -- `⌘`
178
179
180 -- **Empty set**
181
182 example : (setOf (0 < ·) : Set ℤ) ∩ setOf (· < 0) = ∅ := by
183   ext d
184   constructor
185   · rintro ⟨h_pos, h_neg⟩
```

```
186     rw [mem_setOf_eq] at h_neg h_pos
187     rw [lt_iff_not_ge] at h_neg
188     apply h_neg
189     apply le_of_lt
190     exact h_pos
191   · intro h
192     exfalso
193     exact h
194
195
196 -- `⌘`
197
198 -- **§ Indexed unions**
199
200 example {α I : Type} (A : I → Set α) (x : α) : x ∈ ⋃ i, A i
    ↔ ∃ i, x ∈ A i := by
201   -- refine ⟨fun h ↦ ?_, fun h ↦ ?_⟩ -- it is nice to use
    `refine` when dealing with an `↔`
202   -- · rw [mem_iUnion] at h
203   --   exact h
204   -- · rw [mem_iUnion]
205   --   exact h
206   -- -- *Alternative proof*
207   simp -- try also `simp?` to see what has been used
208
209 -- `⌘`
210
211 end Operations
212
213 section Functions
214
215
216 -- # §2: Functions
217
218
219 -- Functions do not natively act on elements of sets: how
    can we fix this code?
220 example (α β : Type) (S : Set α) (T : Set β) (f g : S → β) :
221   -- f = g ↔ ∀ a : α, a ∈ S → f a  = g a := by sorry
222   f = g ↔ ∀ a : α, (ha : a ∈ S) → f ⟨a, ha⟩  = g ⟨a, ha⟩ :=
    by
223   constructor
```

```
224   · intro h
225     rw [h]
226     -- rfl -- *fails*!
227     exact fun _ _ ↦ rfl -- why? Because the whole type is
    not an equality type.
228   · intro h
229     funext x
230     -- exact h --why?
231     apply h -- or exact h _ _
232
233
234 -- `⌘`
235
236 variable (α β : Type)
237
238 -- The **image**
239
240 example : 1 ∈ Nat.succ '' univ := by
241   -- use 0
242   -- constructor
243   -- · trivial
244   -- · rfl
245 -- *An alternative proof*
246   exact ⟨0, ⟨trivial, rfl⟩⟩
247
248 -- We can upgrade a function `f` to a function between sets,
    using its image:
249 example (f : α → β) : Set α → Set β := by
250   intro S
251   exact f '' S
252
253
254 example (α β : Type) (f : α → β) (S : Set α) : S ≠ ∅ → f ''
    S ≠ ∅ := by
255   intro hS hfS
256   apply hS
257   rw [eq_empty_iff_forall_notMem] at hfS ⊢
258   intro x hx
259   replace hfS := hfS (f x)
260   apply hfS
261   use x
262
```

```
263 -- The **preimage**
264
265 example : 2 ∈ Nat.succ ⁻¹' {2, 3} ∧ 1 ∉ .succ ⁻¹' {0, 3} :=
    by
266   constructor
267   · rw [mem_preimage]
268     trivial
269   · intro h
270     rw [mem_preimage] at h
271     trivial
272
273
274 -- `⌘`
275
276 -- **Injectivity**
277
278 example : InjOn (fun n : ℤ ↦ n ^ 2) PositiveIntegers := by
279   intro m hm n hn
280   simp only
281   intro H
282   simp only [Int.pow_succ', Int.pow_zero, Int.mul_one] at H
283   by_cases h_mlt : m < n
284   · exfalso
285     -- have := mul_lt_mul -- we can add a `@` before to let
    it compile anyhow
286     -- have := @Int.mul_lt_mul -- now we see who each
    `a,b,c,d` must be
287     have := Int.mul_lt_mul h_mlt (le_of_lt h_mlt) hm.out
    (le_of_lt hn.out)
288     replace this := ne_of_lt this
289     exact this H
290   by_cases h_nlt : n < m
291   · exfalso
292     exact ((ne_of_lt <| Int.mul_lt_mul h_nlt (le_of_lt
    h_nlt) hn.out (le_of_lt hm.out))) H.symm
293   exact eq_of_le_of_not_lt (le_of_not_gt h_nlt) h_mlt
294
295 end Functions
296
297 section Exercises
298
299 -- **Some exercises**
```

```
300
301 example : 1 ∉ EvenNaturals := by
302   intro h
303   trivial
304   -- tauto
305
306 example : -1 ∉ PositiveIntegers := by
307   intro h
308   trivial
309   -- tauto
310
311 -- Define the set of even, positive numbers
312 def EvenPositiveNaturals : Set PositiveIntegers := by
313   rintro ⟨d, _⟩
314   exact d % 2 = 0
315
316 -- Why does this *fail*? How to fix it?
317 -- example : 1 ∉ EvenPositiveNaturals := sorry
318 /- *Sol.:* Lean complains because `3` is not a term of
    `EvenNaturals`, so it does not make sense
319 to check whether it satisfies a property defined on them. It
    can be made to work by writing -/
320 example : ⟨1, Int.zero_lt_one⟩ ∉ EvenPositiveNaturals := by
321   intro h
322   cases h
323
324 -- Define the set of odd numbers and prove some properties
325 def OddNaturals : Set ℕ := (· % 2 = 1)
326
327 -- Your definition will be right if the following compiles:
328 example : 3 ∈ OddNaturals := by
329   rfl
330
331
332 example (n : ℕ) : n ∈ OddNaturals ↔ n ∉ EvenNaturals := by
333   constructor
334   · intro h h_abs
335     replace h : n % 2 = 1 := h
336     replace h_abs : n % 2 = 0 := h_abs
337     rw [h] at h_abs
338     apply Nat.one_ne_zero
339     exact h_abs
```

```
340   · intro h
341     replace h : ¬ n % 2 = 0 := h
342     rwa [← ne_eq, Nat.mod_two_ne_zero] at h
343
344
345 -- Why does this *fail*?
346 -- example (α : Type) (S : Set α) : subsub = subsub' :=
    sorry
347 /- *Sol.:*  Lean complains because in `subsub'` `P` is
    defined on the type `α` whereas in `subsub`
348 it is defined on the type `↑S`. So this is an equality
    between functions defined on different types,
349 that makes no sense. -/
350
351 -- Try to prove the statement proven before, but without
    using the library
352 example (α : Type) (S T : Set α) (H : S ⊆ T) : T = S ∪ T :=
    by
353   ext
354   constructor
355   · intro h
356     apply Or.intro_right
357     exact h
358   · intro h
359     cases h
360     · apply H
361       assumption
362     · assumption
363
364 example (α : Type) (S T R : Set α) : S ∩ (T ∪ R) = (S ∩ T) ∪
    (S ∩ R) := by
365   ext x
366   refine ⟨fun ⟨h1, h2⟩ ↦ ?_, fun h ↦ ⟨?_, ?_⟩⟩
367   · rcases h2 with hT | hR
368     · exact Or.intro_left _ (⟨h1, hT⟩ : And _ _ )
369     · exact Or.intro_right _ (⟨h1, hR⟩ : And _ _ )
370   · rcases h with ⟨hS, -⟩ | ⟨hS, -⟩ <;> assumption
371   · rcases h with ⟨-, hT⟩ | ⟨-, hR⟩
372     left ; exact hT
373     right ; exact hR
374
375
```

```
376 example (α : Type) (S : Set α) : Sᶜ ∪ S = univ := by
377   ext x
378   constructor
379   · intro
380     trivial
381   · intro
382     by_cases hx : x ∈ S
383     · apply Or.intro_right
384       exact hx
385     · exact Or.intro_left _ hx
386
387 -- For this, you can try `simp` at a certain
    point...`le_antisymm` can also be useful.
388 example : (setOf (0 ≤ ·) : Set ℤ) ∩ setOf (· ≤ 0) = {0} :=
    by
389   ext
390   simp only [mem_inter_iff, mem_setOf_eq, mem_singleton_iff]
391   constructor
392   · intro h
393     exact (le_antisymm h.1 h.2).symm
394   · intro h
395     rw [h]
396     exact ⟨le_refl _, le_refl _⟩
397
398
399 -- Using your definition of `OddNaturals` prove the
    following:
400 example : EvenNaturals ∪ OddNaturals = univ := by
401   ext x
402   simp only [mem_union, mem_univ, iff_true] -- to be
    obtained by typing `simp?`
403   by_cases hx : x % 2 = 0
404   · apply Or.inl
405     exact hx
406   · rw [← ne_eq, Nat.mod_two_ne_zero] at hx
407     apply Or.inr
408     exact hx
409
410
411 -- **§** A bit of difference, inclusion and intersection
412
```

```
example (α : Type) (S T : Set α) (h : T ⊆ S) : T \ S = ∅ :=
by
  ext
  exact ⟨fun ⟨hT, hnS⟩ ↦ hnS <| h hT, fun _ ↦ by trivial⟩


example (α : Type) (S T R : Set α) : S \ (T ∪ R) ⊆ (S \ T) \
R := by
  intro x ⟨hxS, hxnTR⟩
  rw [mem_diff]
  rw [mem_union, not_or] at hxnTR
  exact ⟨⟨hxS, fun h ↦ hxnTR.1 h⟩, hxnTR.2⟩
-- *An alternative tactic* replacing this `exact`: longer
but easier to read:
  -- constructor
  -- · constructor
  --   · exact hxS
  --   · exact hxnTR.1
  -- exact hxnTR.2


-- Indexed intersections work as indexed unions (_mutatis
mutandis_)
example {α I : Type} (A B : I → Set α) : (⋂ i, A i ∩ B i) =
(⋂ i, A i) ∩ ⋂ i, B i := by
  ext x
  simp only [mem_inter_iff, mem_iInter]
  constructor
  · intro h
    constructor
    · intro i
      exact (h i).1
    intro i
    exact (h i).2
  rintro ⟨h1, h2⟩ i
  constructor
  · exact h1 i
  exact h2 i


example {α I : Type} (A : I → Set α) (S : Set α) : (S ∩ ⋃ i,
A i) = ⋃ i, A i ∩ S := by
```

```
449   ext x
450   simp only [mem_inter_iff, mem_iUnion]
451   constructor
452   · rintro ⟨xs, ⟨i, xAi⟩⟩
453     exact ⟨i, xAi, xs⟩
454   rintro ⟨i, xAi, xs⟩
455   exact ⟨xs, ⟨i, xAi⟩⟩
456
457 open Function
458
459 variable (α β : Type) (f : α → β)
460
461
462 -- The range is not *definitionally equal* to the image of
    the universal set:
463 example : range f = f '' univ := by
464   ext x
465   refine ⟨fun h ↦ ?_ , fun h ↦ ?_⟩
466   · rw [mem_range] at h
467     obtain ⟨y, hy⟩ := h
468     use y
469     constructor
470     · trivial
471     · exact hy
472   · rw [mem_range]
473     obtain ⟨y, hy⟩ := h
474     use y
475     exact hy.2
476
477
478 -- Why does this code *fail*? Fix it, and then prove the
    statement
479 -- example (N : OddNaturals) : N ∈ Nat.succ ''
    (EvenNaturals) :=
480 -- It *fails* because `N` is of the wrong type. `N.1` is a
    `ℕ`, and the following code compiles:
481 example (N : OddNaturals) : N.1 ∈ Nat.succ '' (EvenNaturals)
    := by
482   rcases N with ⟨n, hn⟩
483   have hn' := hn.out
484   rw [mem_image]
485   match n with
```

```
  | 0 => trivial
  | m+1 =>
    rw [Nat.succ_mod_two_eq_one_iff] at hn'
    use m
    constructor
    · exact hn'
    · exact Nat.succ_eq_add_one _


-- Why does this code *fail*? Fix it, and then prove the statement
-- example (N : OddNaturals) :  N ∈ Nat.succ ⁻¹' (EvenNaturals) := by
-- It *fails* because `N` is of the wrong type. `N.1` is a `ℕ`, and the following code compiles:
example (N : OddNaturals) :  N.1 ∈ Nat.succ ⁻¹' (EvenNaturals) := by
  rcases N with ⟨n, hn⟩
  rw [mem_preimage]
  have hn' := hn.out
  by_cases hnz : n = 0
  · rw [hnz] at hn'
    trivial
  · replace hnz := Nat.exists_eq_succ_of_ne_zero hnz
    obtain ⟨m, hm⟩ := hnz
    rw [hm] at hn'
    rw [Nat.succ_mod_two_eq_one_iff] at hn'
    change n.succ % 2 = 0 --`show` changes the goal to something definitionally equivalent
    grind
    -- *Alternative proof* if you forgot about `grind`:
    -- rw [hm]
    -- simp only [Nat.succ_eq_add_one]
    -- rwa [add_assoc, Nat.add_mod_right]


-- Not every `n : ℕ` is the successor or something...
example : range Nat.succ ≠ univ := by
  intro h
  rw [Set.eq_univ_iff_forall] at h
  replace h := h 0
  have := Nat.succ_ne_zero (Exists.choose h)
```

```
523   exact this (Exists.choose_spec h)
524
525
526
527 /- The following is a *statement* and not merely the
    *definition* of being injective;
528   prove it. -/
529 example : Injective f ↔ InjOn f univ := by
530   refine ⟨fun h ↦ ?_, fun h ↦ ?_⟩
531   · intro a ha b hb H
532     apply h H
533   · intro a b H
534     exact h (trivial) (trivial) H
535
536
537
538 /- With the obvious definition of surjective, state and
    prove the following result: the
539  complement `Sᶜ` is referred to with the abbreviation
    `compl` in the library -/
540 example : Surjective f ↔ (range f)ᶜ = ∅ := by
541   refine ⟨fun h ↦ ?_ , fun h ↦ ?_⟩
542   · rw [Set.compl_empty_iff]
543     ext x
544     constructor
545     · intro H
546       trivial
547     · intro H
548       obtain ⟨y, hy⟩ := h x
549       use y
550   · intro x
551     rw [Set.compl_empty_iff] at h
552     rw [eq_univ_iff_forall] at h
553     replace h := h x
554     exact h
555
556 end Exercises
557
```